Bilel Bouallouche
~~26109410~~

Université de Rennes

Notice d'utilisation

Nous avons développé en C un programme capable de chiffrer en utilisant l'algorithme AES. Le programme a les fonctionnalités suivantes:

- chiffrement et déchiffrement pour des clés quelconques de tailles 128, 192 ou 256 bits.
- chiffrement et déchiffrement pour les modes ECB, CBC, CFB.
- benchmarking sur le chiffrement AES-128-ECB sur le fichier alice.sage.

```
bbouallouche@b40-011m04:~/Bureau/crypto/aes$ ./bin/aes -h
Usage:  aes [options] [path]
Crypt or decrypt a file using AES
Options:
        -h, --help                              Display this help message
        -e [FILE], --encrypt [FILE]             Encrypt the file given as input and following the key size and mode set
        -d [FILE], --decrypt [FILE]             Decrypt the file given as input and following the key size and mode set
        -o [FILE], --output [FILE]              Set the output file
        -b, --benchmark                         Launch a benchmarking test
        -k, --key=hexstring                     Set the key (the size must be 128 or 192 or 256) [default = 000102030405060708090a0b0c0d0e0f]
        -m, --mode=MODE                         Set the mode (ECB, CBC, CFB) [default = ECB]
```

En plus du programme principal, nous avons développé un script shell (test.sh) afin de tester notre programme. Ce script s'exécute en deux parties:

- On lance le benchmark.
- On chiffre avec notre programme le fichier alice.sage en AES-128 pour chacun des trois modes. On obtient ainsi trois chiffrés distincts. Le script se sert ensuite d'OpenSSL (ainsi que quelques utilitaires Linux)[1], pour construire les déchiffrés associés et comparer qu'ils sont bien identiques aux fichiers de départ alice.sage.

Avant toute exécution, il faudra se placer dans le répertoire racine du projet et lancer la compilation grâce à la commande *make*.

Description de l'implémentation

Pour l'implémentation, on a séparé en quatre modules distincts ce projet:

1. un module aes.c/.h, qui est le moteur de calcul principal et suivant la description du NIST.
2. un module key.c/.h, où l'on fait le key scheduling décrit par le NIST.
3. un module mode.c/.h, où l'on appelle les fonctions de chiffrement et déchiffrement par bloc du module aes dans le bon ordre pour implémenter les différents modes d'opérations.
4. un module io.c/.h, dont le rôle est simplement l'écriture et la lecture de fichiers.

[1] J'ai appris à me servir d'OpenSSL lors du CTF Hack'Lantique, pour une épreuve sur RSA. La syntaxe du script s'est faite laborieusement avec l'aide de Google

Ainsi, le code est correctement compartimenté. Les deux premiers modules sont conformes aux spécifications du NIST et n'ont aucun autre rôle que de les respecter.
Les autres modules, eux, sont plus libres. Ainsi, le module mode suit simplement les descriptions vues en cours et io est une simple redite des quelques fonctions de gestion de fichiers qu'on a vu en cous de LLP.
Pour le module aes, il s'agissait de rester au plus proche possible des quelques astuces qu'on a pu voir en TD et en TP pour maximiser la vitesse de calcul.
Ainsi, on avait vu qu'on pouvait :

- se servir de tableaux pré-calculés, ce que j'ai décidé de faire pour implémenter les opérations SubBytes et son inverse. En effet, faire le calcul aurait été calculatoirement plus lourd que simplement lire deux sbox.
- on avait vu comment optimiser MixColumns et son inverse (qui ne consiste qu'en deux applications linéaires) grâce à la nature de la multiplication dans le corps de Galois définissant le corps où l'on fait les calculs d'AES. Nous avions ainsi défini une opération "xtime" qui optimisait la multiplication par 2 comme étant au mieux un simple décalage de bits et au pire un décalage de bits suivi d'un XOR par une constante.
- L'opération MixColumns et son inverse nécessite plus que des multiplications par 2, mais aussi par 0x03, 0x09, 0x0B, 0x0D et 0x0E. On s'est donc inspiré de l'algorithme square and multiply vu en classe pour se ramener a l'opération xtime, optimisé, et a des XOR bits à bits, eux aussi optimisés.
  Par exemple, on veut multiplier une quantité t par $0x0E$ (14) :

$$\begin{aligned} t \times 0x0E &= t \times (8+4+2) \\ &= t \times 8 \oplus t \times 4 \oplus t \times 2 \\ &= t \times 2^3 \oplus t \times 2^2 \oplus t \times 2 \\ &= xtime(xtime(xtime(t))) \oplus xtime(xtime(t)) \oplus xtime(t) \end{aligned}$$

  Ainsi, on ne fait jamais aucune multiplication directement, on ne fait que des xor et des décalages de bits.

Difficultés rencontrées

Les trois difficultées ont été

- l'algorithme de key schedule. Il est relativement simple une fois fait, mais la spécification du NIST est extrêmement dure à lire sur cette section.
- écrire un code permettant de généraliser les algorithmes à toutes tailles de clés.
  Il m'a fallu être prévoyant, réfléchir au design de mes fonctions, réfléchir à certaines macros du préprocesseur en lisant correctement la spécification du NIST.
- le mode CFB m'a donné, en terme de compréhension de l'algorithme, un peu de fil à retordre.
- La gestion des erreurs. Disons qu'un malloc échoue lors du padding d'un message.
  Comment gérer ceci ? Cela nous force à réfléchir en amont aux retours possibles de nos fonctions, à quel moment simplement prévenir l'utilisateur dans le terminal, comment quitter proprement la fonction de chiffrement qui a appelé la fonction de padding en cas d'échec, comment gérer cette cascade de mise en échec de l'algorithme.

Pistes d'amélioration

- Rajouter les autres modes vu en cours, notamment GCM.
- Actuellement, le chiffrement et déchiffrement 100 fois d'affilées prends environ 1 seconde sur les ordinateurs de l'ISTIC et près de 5 secondes sur mon ordinateur personnel. Je pense que c'est largement améliorable.
- Je pense que mon code n'est pas à temps constant (notamment la fonction pour le padding ou xtime).